

Web Basic

Dockerfile과 Docker Compose

환경변수

환경변수

환경 변수란

- 프로세스가 동작할 때 참조할 수 있는 변수다.
- 이미지를 빌드할 때 각 레이어에서 참조할 수 있다.
- 컨테이너가 실행되는 동안 참조할 수 있다.
- 환경 변수를 통해 서버의 포트, 컨텍스트 경로 등 동작에 영향을 미치는 다양한 정보를 동적으로 관리할 수 있다.
- Dockerfile의 ENV 명령어를 통해 환경 변수를 설정할 수 있다.
- 부모 이미지의 ENV는 자식 이미지에 상속된다.
 - 즉, 부모 이미지에서 선언된 ENV는 내 이미지에서도 유효하다.

ENV 사용법

```
ENV <key>=<value> ...
```

- <key>는 환경 변수의 이름, <value>는 값이다.

```
ENV VAR_A="A"
```

```
ENV VAR_A=A
```

- 쌍따옴표는 무시된다.

```
ENV VAR_A="Hello \  
World"
```

- \ (backslash)를 통해 여러 줄로 작성할 수 있다.

```
ENV VAR_A="A" VAR_B="B"
```

- 공백을 기준으로 여러 개의 환경 변수를 정의할 수 있다.

다른 시점에 ENV 값 정의하기

- ARG 명령어와 --build-arg 옵션을 통해 이미지를 빌드하는 시점에 정의할 수 있다.

```
# docker build --build-arg VAR_A_FROM_ARG=A
ARG VAR_A_FROM_ARG
ENV VAR_A=${VAR_A_FROM_ARG}
```

- -e 옵션을 통해 컨테이너 실행 시점에 정의할 수 있다.

```
docker run -e VAR_A=A my-container
```

- --env-file 옵션을 통해 컨테이너 실행 시점에 파일로 정의할 수 있다.

```
docker run --env-file .env my-container
# .env 파일
VAR_A=A
VAR_B=B
```

`${VAR}`와 `$VAR`

- VAR라는 환경 변수를 참조하기 위해 `${VAR}`와 `$VAR` 모두 사용 가능하다.
- 두 가지 이유로, `${VAR}` 표기법이 더 선호된다.
- 해당 변수와 다른 문자열을 붙여 사용하고자 할 때 `${VAR}` 표기만 가능하다.

```
# VAR 값이 v인 경우
${VAR}_A  # v_A
$VAR_A    # ${VAR_A}로 해석됨
```

- 조건부 표기가 가능하다.

```
${VAR:-defaultValue}  # VAR 값이 있으면 VAR, 없으면 defaultValue
${VAR:+defaultValue}    # VAR 값이 있으면 defaultValue, 없으면 빈 문자열
```

유의사항

- 접두사를 활용하여 시스템에서 활용하는 환경 변수를 덮어쓰지 않도록 정의한다.
- 이미지 빌드 시점에만 활용할 변수라면 ARG 명령어를 사용하는 것이 좋다.
- ENV 를 선언할 때 "="는 생략 가능하나, 구문의 오류 가능성이 있고, 추후 삭제될 수 있다.
- 따라서 "="를 명시적으로 사용하는 표기법을 따르는 것이 바람직하다.

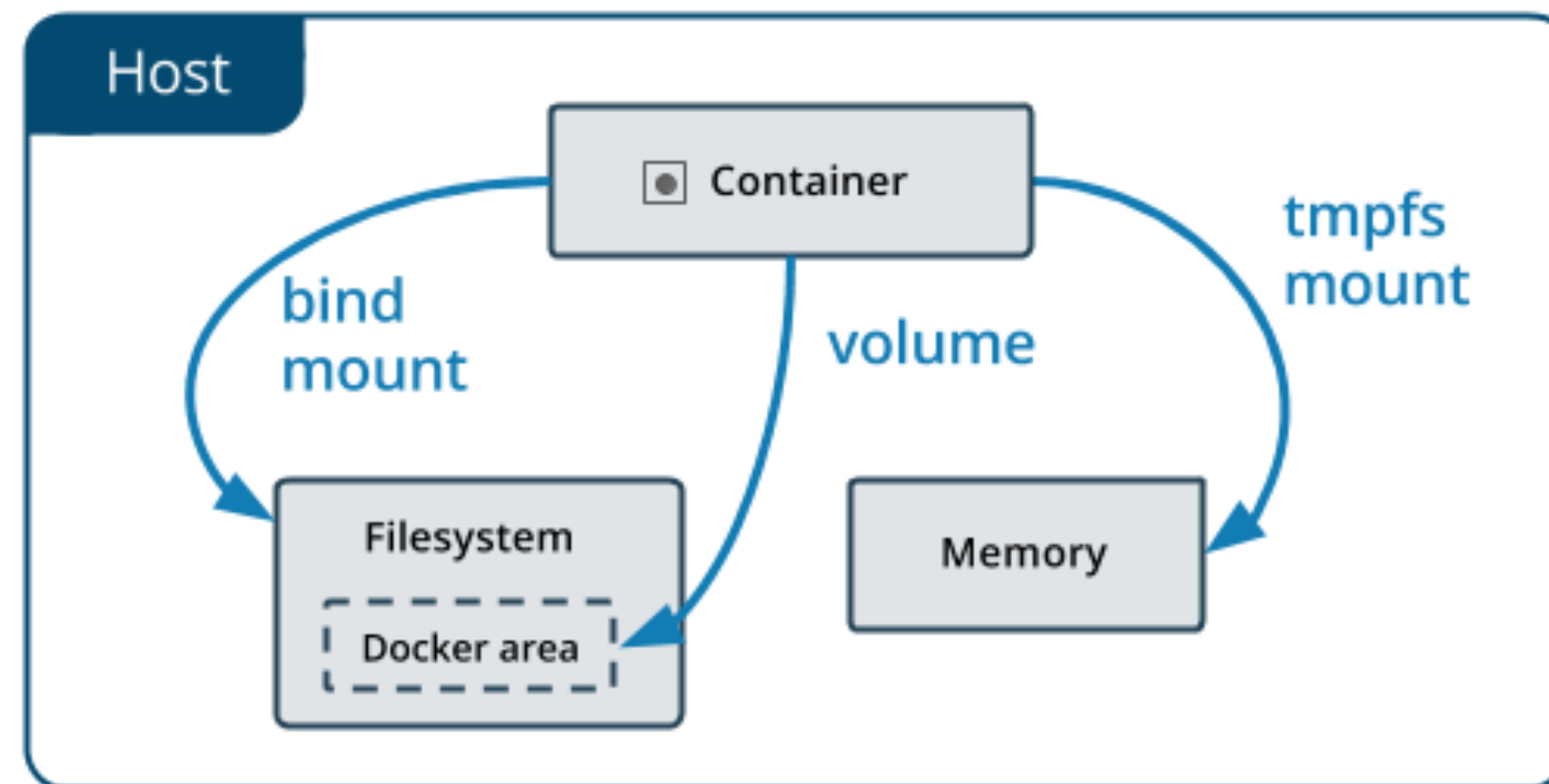
부부

컨테이너의 휘발성

- 컨테이너가 실행되는 동안 쓰여진 데이터는 컨테이너가 삭제되면 같이 사라진다.
- 만약 mysql 컨테이너를 사용했다면, 컨테이너를 삭제하는 순간 모든 데이터베이스 정보가 삭제된다.
- 따라서 계속 유지해야 할 정보가 있다면, 컨테이너 외부에 저장해야 한다.
- 이와 같이 데이터를 영구적으로 저장하는 행위를 “영속화” 한다고 한다.

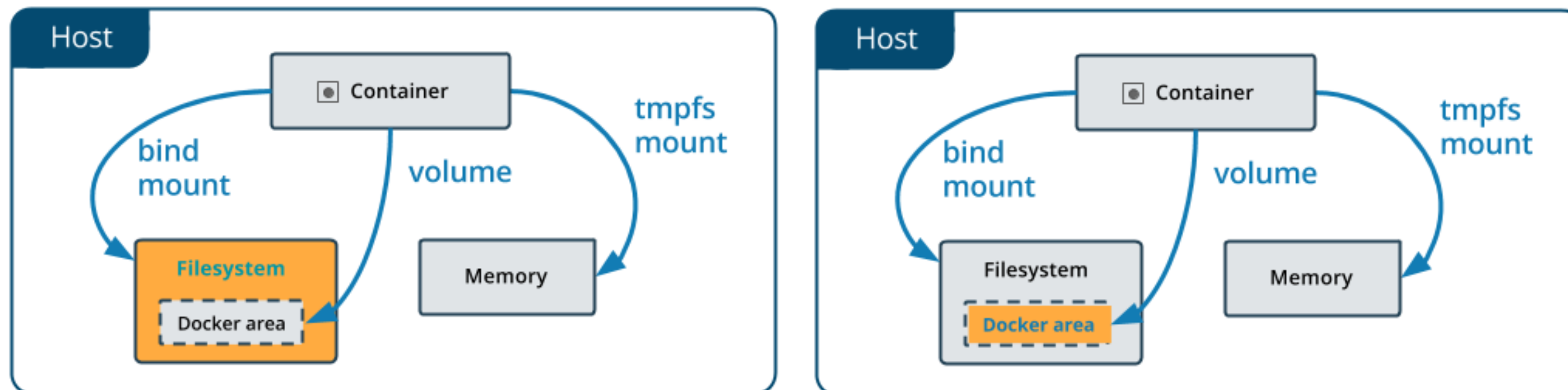
데이터를 영속화하기

- Docker 엔진은 두 가지 방법(bind mount, volume)으로 영속화 기능을 제공한다.
 - tmpfs mount 방식은 영속화 방식이 아니므로 생략한다.



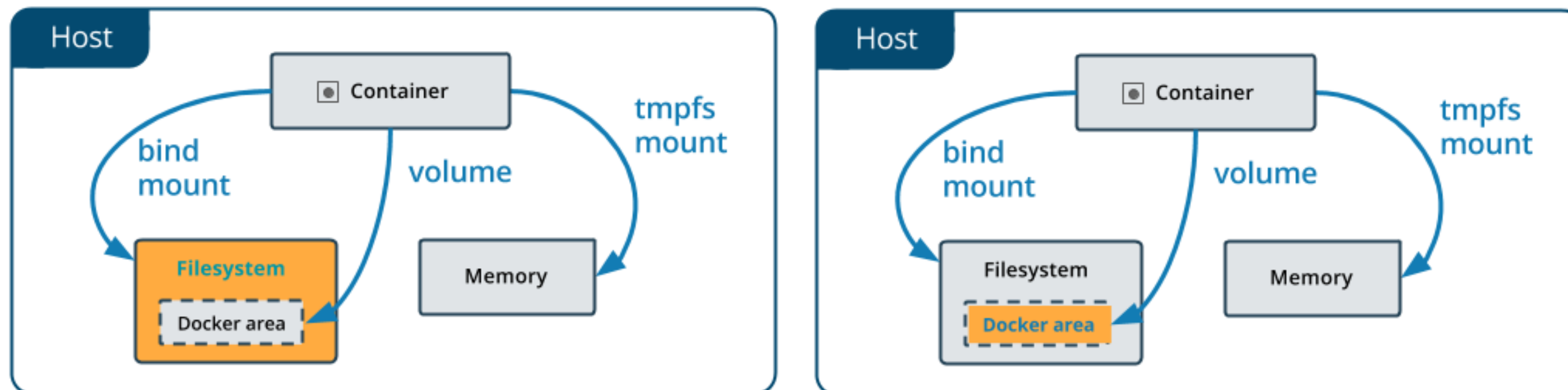
데이터를 영속화하기

- Bind mount는 호스트 머신의 FileSystem를 직접 마운트*하는 방식이다.
 - 이 방식은 호스트 머신의 FileSystem에 영향을 많이 받을 수 있다.
- Volume은 Docker에서 관리하는 특정 공간에 마운트하는 방식이다.
 - 이 방식은 호스트 머신의 FileSystem에 비교적 영향을 많이 받지 않는다.



데이터를 영속화하기

- Volume 방식이 Bind mount 방식에 비해 성능상 우수하다.
- Volume 방식은 Docker CLI, API를 통해 관리할 수 있다.
- 이 밖에도 Volume은 Bind mount 방식에 비해 이점이 많으며, 공식 문서에서도 더 권장되는 방식이다.
- 따라서 특별한 이유가 없다면, Volume 방식을 고려하자.



Dockerfile VOLUME

```
VOLUME /some/path /other/path  
VOLUME ["/some/path", "/other/path"]
```

- 마운트 할 컨테이너의 경로를 표기한다.
- JSON array 형식으로 표기할 수 있다.
- 컨테이너가 실행되면 volume이 생성되고 컨테이너 내 해당 경로에 마운트 포인트가 생성된다.
 - volume과 컨테이너 내 마운트 포인트는 모두 자동으로 생성된다.

```
→ docker volume ls  
DRIVER      VOLUME NAME  
local       8209cf3c2cf960d878a23b40453d0cb78fe9be18326d59ac53ab3c846584b920  
local       233069e2040d15f877db25f5f0a98df845ecfe7fda3c393f2cf423d44113193e
```

- 컨테이너 단위로 생성되며, 이름이 해시값이기 때문에 관리하기가 어려울 수 있다.

docker run -v 옵션

```
→ docker run -v my-vol:/some/path my-container
```

- 콜론(:)을 기준으로 왼쪽에 volume의 이름을, 오른쪽에 컨테이너 내 마운트 포인트를 기술한다.
- VOLUME 방식과 마찬가지로 volume과 마운트 포인트는 없는 경우 자동으로 생성된다.

```
→ docker volume ls  
DRIVER      VOLUME NAME  
local       my-vol
```

- 마운트 포인트를 지정하지 않으면 볼륨 이름의 경로가 자동으로 생성된다.

```
→ docker run -v my-vol my-container
```

docker run --mount 옵션

```
→ docker run --mount type=volume,src=my-vol,dst=/some/path my-container
```

- 속성(type, source, destination 등)에 대해 <key>=<value> 형태로 기술한다.
- type 속성 값은 "volume"이다.
- VOLUME 방식과 마찬가지로 volume과 마운트 포인트는 없는 경우 자동으로 생성된다.

```
→ docker volume ls  
DRIVER      VOLUME NAME  
local       my-vol
```

- -v 옵션과 기능적으로 같다.
- 상세한 옵션을 활용하고자 할 때 사용하는 것이 좋다.

docker run -v 옵션

```
→ docker run -v /some/host/path:/some/container/path my-container
```

- 콜론(:)을 기준으로 왼쪽에 호스트 머신의 파일 또는 디렉토리, 오른쪽에 컨테이너 내 마운트 포인트를 기술한다.
- 호스트 머신의 경로가 없거나 마운트 포인트는 없는 경우 자동으로 생성된다.
- Volume 은 별도로 생성되지 않는다.

docker run --mount 옵션

```
→ docker run --mount type=bind,src=/some/host/path,dst=/some/path my-container
```

- 속성(type, source, destination 등)에 대해 <key>=<value> 형태로 기술한다.
- type 속성 값은 "bind"이다.
- 마운트 포인트는 없는 경우 자동으로 생성된다.
- 단, -v 옵션과 다르게 호스트 머신의 경로가 없는 경우 에러가 발생한다.

```
→ docker run --mount type=bind,src=/some/host/not-exist/path,dst=/some/path my-container
```

```
docker: Error response from daemon: invalid mount config for type "bind": bind source path does not exist: /host_mnt/some/host/not-exist/path.  
See 'docker run --help'.
```

Volume vs Bind Mount 비교표

	Bind Mount	Volume
저장 위치	유저가 정한 호스트 컴퓨터의 특정 디렉토리	도커가 정한 호스트 컴퓨터의 특정 디렉토리
관리 주체	유저	도커
접근성	탐색기에서 바로 열고 수정 가능	도커를 통해서 접근하는 것이 원칙
주요 용도	소스 코드 공유, 실시간 개발	데이터베이스 파일, 로그, 설정값 보존

네트워크

도커의 네트워크 이해하기

- 다수의 컨테이너를 다룰 때 가장 신경써야 할 부분은 네트워크 통신이다.
- 컨테이너는 기본적으로 격리된 자원이다.
- 따라서 다른 컨테이너와 통신하기 위해선 별도의 수단이 필요하다.
- 도커에서 제공하는 네트워크 인프라, 즉, 도커 네트워크를 통해 컨테이너 간 통신을 제어할 수 있다.

도커 네트워크의 주요 개념

- 컨테이너 간 IP 또는 DNS로 통신할 수 있는 격리된 네트워크를 제공한다.
- 드라이버를 통해 여러가지 네트워크를 정의할 수 있으며, 기본적으로 6가지 드라이버를 제공한다.
 - bridge, host, overlay, ipvlan, macvlan, none
- 도커 엔진에는 기본적으로 3가지 드라이버 기반의 네트워크가 생성되어 있다.

```
→ docker network ls
```

NETWORK ID	NAME	DRIVER	SCOPE
7ee0351b3249	bridge	bridge	local
a4d754ad2ce3	host	host	local
56277379796c	none	null	local

도커 네트워크의 주요 개념

- 도커 CLI 명령어를 통해 네트워크를 생성할 수 있다.

```
docker network create elice-network
```

- 컨테이너를 실행하면 기본 제공된 bridge 네트워크를 사용한다.
- Bridge 네트워크는 기본적으로 IP 주소를 통한 컨테이너간 통신을 지원한다.
- 추가로 사용자 정의 Bridge 네트워크는 DNS와 컨테이너 이름을 통한 통신까지 지원한다.



두 개의 nginx 컨테이너 실행



```
docker run -d --name nginx1 nginx  
docker run -d --name nginx2 nginx
```

컨테이너 간 통신을 확인하기 위해 두 개의 컨테이너를 생성한다.

별도의 네트워크 설정을 하지 않으면 bridge 네트워크에 할당된다.



각 컨테이너의 IP 주소 확인



```
# nginx1의 IP 확인
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' nginx1

# nginx2의 IP 확인
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' nginx
```

각 컨테이너에 할당된 사설 IP 주소를 확인한다.



IP를 통한 통신 테스트 (성공)



```
# 1. nginx1 내부 접속  
docker exec -it nginx1 bash  
  
# 2. nginx2의 IP로 curl 요청 (위에서 확인한 IP 입력)  
curl [nginx2의_IP_주소]
```

nginx1 컨테이너 내부로 접속하여
nginx2의 IP 주소로 curl* 요청을 보내본다.



컨테이너 이름을 통한 통신 테스트 (실패)



```
# nginx1 내부에서 실행  
curl nginx2
```

이번에는 IP 대신 컨테이너 이름을 사용하여 통신을 시도해본다.



기본 bridge 네트워크 통신 정리

- 컨테이너는 기본적으로 bridge 네트워크에 속한다.
- 기본 Bridge 네트워크의 컨테이너는 IP 주소를 통해 통신할 수 있다.
- 반면, 기본 bridge 네트워크는 내장 DNS 서버가 제공되지 않아 컨테이너 이름을 해석할 수 없으므로, 통신할 수 없다.

사용자 정의 네트워크 생성 및 실행



```
docker network create elice-network
```



```
# 컨테이너 1 (web1)
```

```
docker run -d --name web1 --network elice-network nginx
```

```
# 컨테이너 2 (web2)
```

```
docker run -d --name web2 --network elice-network nginx
```

먼저 컨테이너들이 소속될 전용 네트워크를 생성 및 실행한다.

컨테이너 이름(DNS)을 통한 통신 테스트 (성공)

```
● ● ●  
  
# 1. web1 내부 접속  
docker exec -it web1 bash  
  
# 2. 컨테이너 이름(web2)으로 직접 curl 요청  
curl web2
```

이제 IP 주소를 몰라도 이름만으로 통신이 되는지 확인한다.

web1에 접속하여 web2에게 curl 요청을 보낸다.

컨테이너 간 통신 정리

- `docker run --network elice-network` 옵션을 통해 network를 설정할 수 있다.
- 같은 사용자 정의 네트워크(`elice-network`)에 연결된 두 컨테이너는 DNS, 컨테이너 이름으로도 통신이 가능하다.

Dockerfile과 Docker Compose

Docker Compose 구성 및 기본 문법

다중 컨테이너

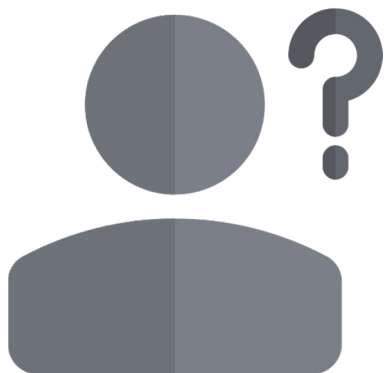
다중 컨테이너의 개념

여러 개의 컨테이너가 하나의 애플리케이션을 구성하는 구조

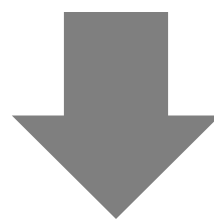
- 과거에는 한 서버(혹은 한 컨테이너)에 애플리케이션의 모든 기능을 넣어 사용하였으나, 현대 시스템에서는 기능을 여러 개의 독립된 컨테이너로 분리
- 예) 웹 서버 컨테이너 (Frontend), API 컨테이너 (Backend), DB 컨테이너 등
- 독립된 컨테이너들이 네트워크로 서로 통신하며 하나의 서비스처럼 동작



다중 컨테이너 필요성



왜 다중 컨테이너가 필요한가요?

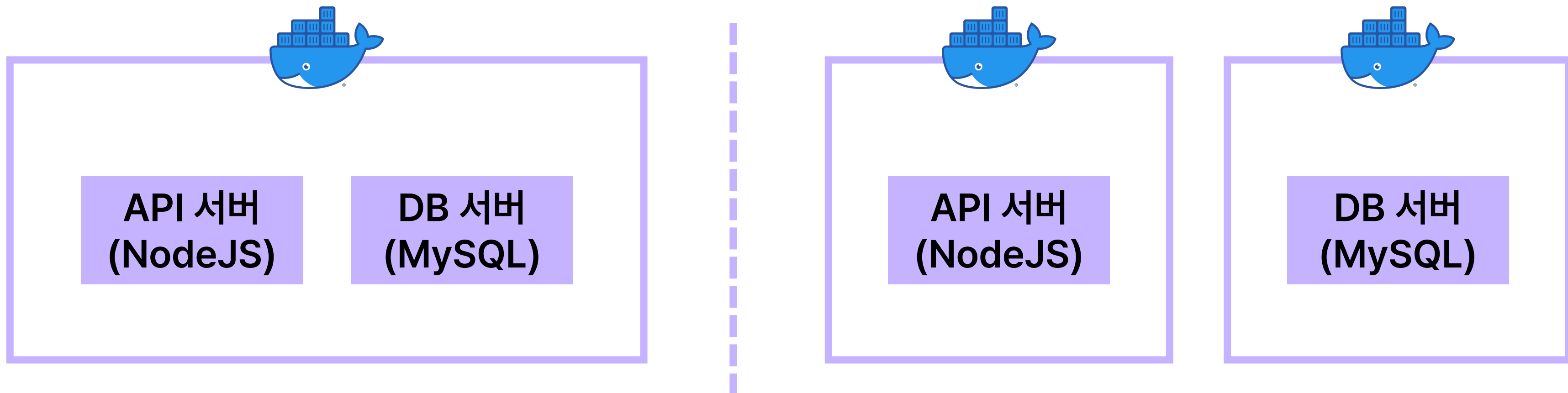


- ✓ **기능별로 분리 (Microservices 아키텍처):** 변경/배포가 독립, 유지보수 쉬움, 장애 국소화
- ✓ **스케일링 단위가 다름:** 각 기능의 부하량이 다르기 때문에 따로 확장
- ✓ **언어/환경의 독립성:** 각 컨테이너는 자기만의 런타임을 가질 수 있음
(Frontend → Node.js, Backend → Python, DB → MySQL)

다수의 컨테이너를 사용하는 이점

프로세스 실행 주기의 독립

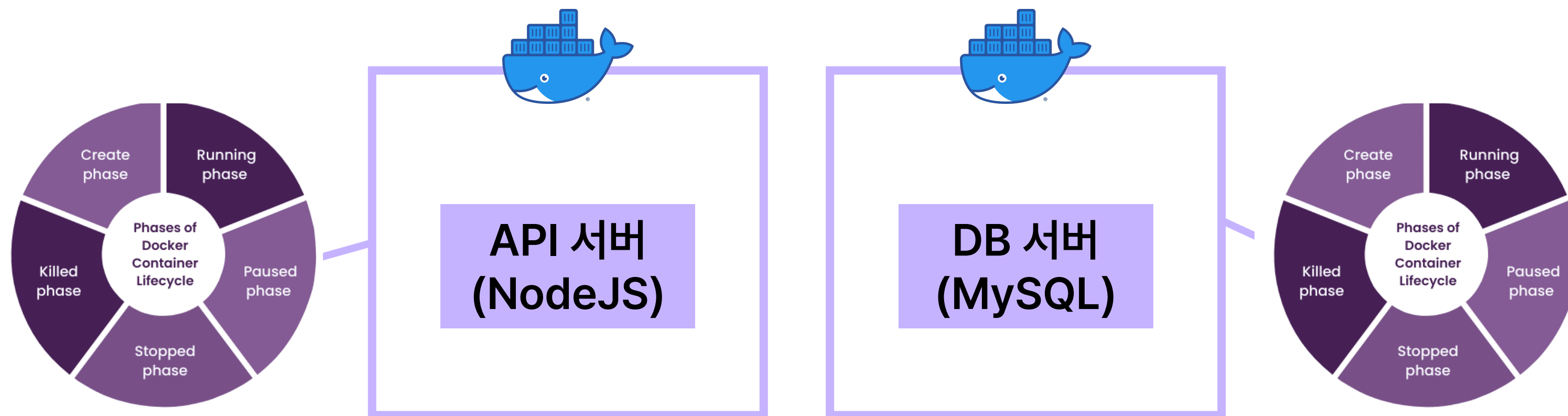
- 만약 단일 컨테이너로 구성한다면 ENTRYPOINT를 작성하기 까다로움
- 이 의미는 다수의 프로세스를 한 시점에 모두 다루는 것이 까다롭다는 것을 의미
- 다수의 컨테이너라면, 프로세스의 실행 사이클이 독립적이므로, 비교적 다루기 편리함



다수의 컨테이너를 사용하는 이점

독립된 관리

- 만약 API 서버 개발팀과 DB 관리팀이 나누어져 있다면, 각 팀은 각각의 미션에 집중하는 것이 좋음
- 단일 컨테이너라면 버전 관리, 배포, 모니터링과 로깅 등 두 팀간 협의할 사항이 훨씬 증가
- 다수의 컨테이너라면, 독립적으로 관리할 수 있기 때문에 효율적인 협업이 가능



다수의 컨테이너를 사용하는 이점

효율적인 자원 할당

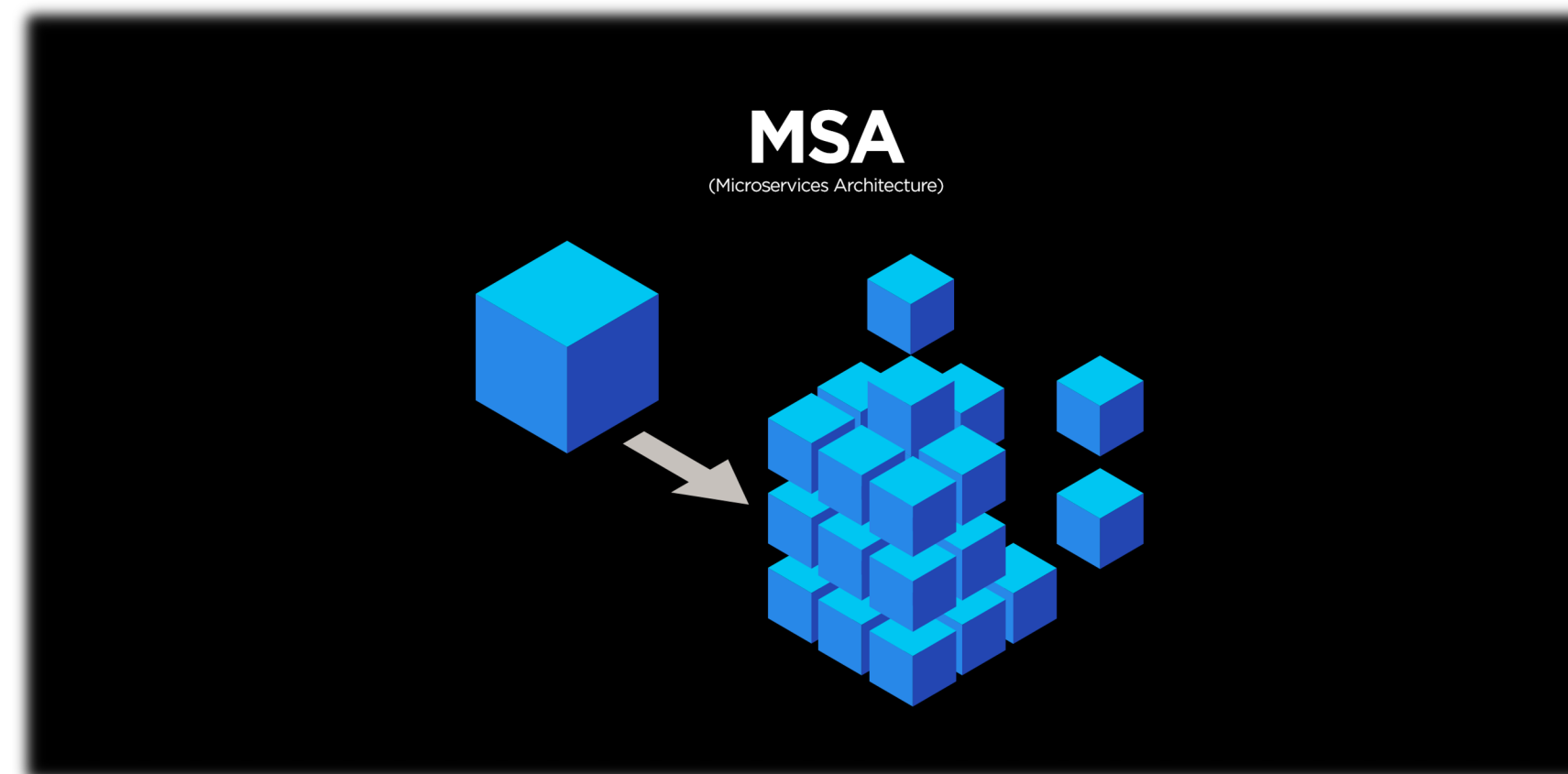
- 컨테이너는 격리된 리소스의 집합
- 만약 API 서버의 부하가 많아져 자원을 늘려야 한다고 가정 → 단일 컨테이너라면 API 서버의 자원만 집중적으로 늘리기 까다로움



다수의 컨테이너를 사용하는 이점

MSA

- Micro Service Architecture는 가장 현대적인 서비스 아키텍처
- MSA 사상을 적용하기 위해선 단일 컨테이너 만으로는 불가능
- MSA를 적용하고, 이점을 가져가기 위해서 다수의 컨테이너로 구성하는 것이 바람직





다중 컨테이너를 사용해야 하는 이유

Dockerfile로 다중 컨테이너를 정의할 때의 불편함

해결 방법

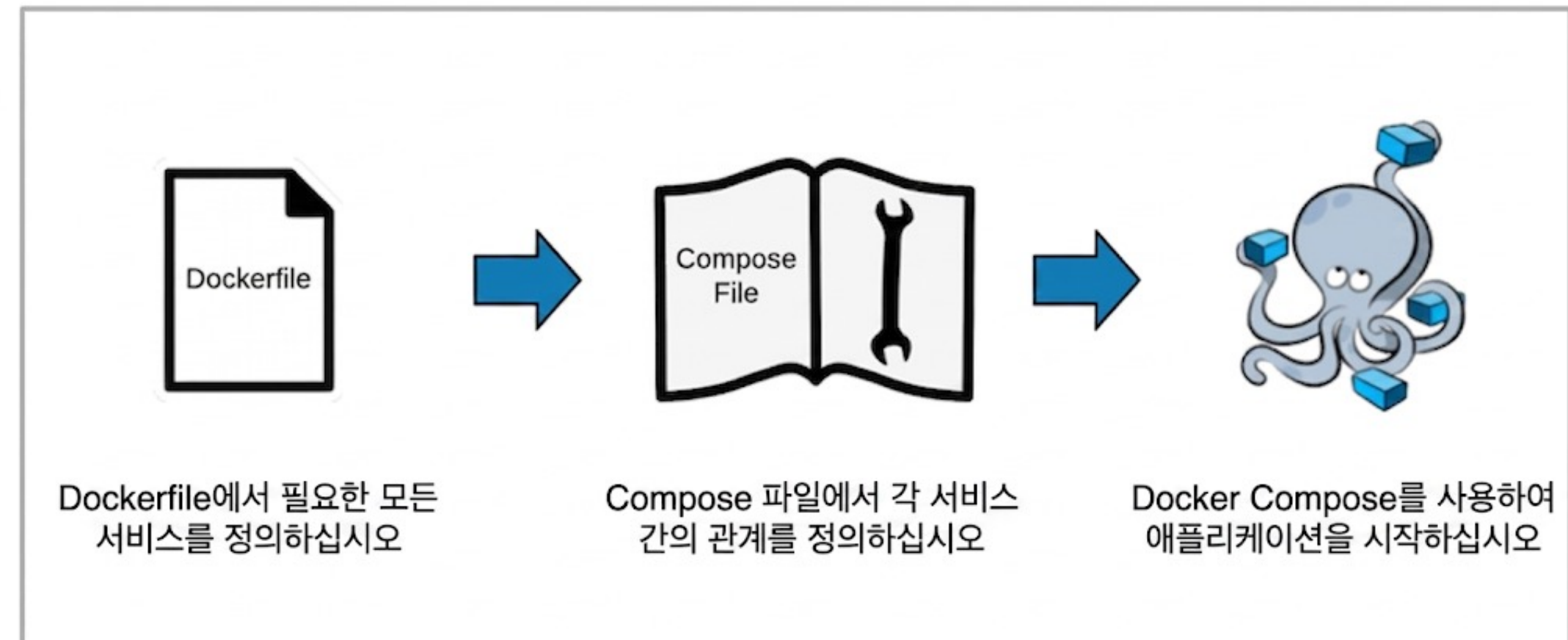
- 앞서 구현한 다중 컨테이너를 팀원과 공유한다고 생각해보자.
- 컨테이너, 실행 시 옵션, 네트워크, 환경 변수 등 설명할 것들이 다소 복잡하다.
- 만약 하나의 파일로 이 모든 것들을 설명할 수 있다면 얼마나 좋을까?
- Docker Compose가 이 불편함을 해결해 줄 수 있다.

Docker Compose 구성 및 기본 문법

Docker Compose의 개념

다중 컨테이너를 정의하고 관리할 수 있도록 도와주는 툴

- yml 확장자 파일로 작성
- 보통 docker-compose.yml를 파일명으로 사용
- Dockerfile를 정의할 때 Dockerfile을 파일명으로 사용하는 것과 비슷
- yml 파일 하나로 이미지 빌드, 컨테이너 실행, 도커 컴포넌트 (네트워크, 볼륨 등)를 정의할 수 있음





Docker Compose 소개

Docker Compose 작성 예시

다중 컨테이너를 docker-compose.yml 파일로 정의한 예시

```
services:
  my-app:
    image: myapp.db
    container_name: my-app
    build:
      context: /some-path/my-app
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - MY_DB_HOST=my-db
    networks:
      - elice-network
```

```
my-db:
  image: mysql: 9.6.0
  container_name: my-db
  volumes:
    - /some-path/compose/init.sql:/docker-entrypoint-initdb.d/init.sql
  ports:
    - "3306:3306"
  env_file:
    - .mysql-env
```

*version은 Docker Compose v2부터 무시되는 deprecated 속성이므로 생략한다.

실행 및 생성 시 사용하는 명령어

docker compose up	포그라운드 실행 모드로 터미널에 로그가 실시간 출력됨
docker compose up -d	백그라운드 실행 모드로 터미널 사용 가능 (권장)
docker compose up --build	이미지를 새로 빌드한 후 컨테이너 실행 (소스코드 수정 후 사용)

예 시) docker compose up -d --build

설정 파일을 기반으로 Dockerfile을 새로 빌드하여 컨테이너를 백그라운드에서 생성 및 실행

상태 확인 및 모니터링 명령어

<code>docker compose ps</code>	현재 컨테이너 목록 및 상태 확인
<code>docker compose logs</code>	서비스 로그 출력
<code>docker compose logs -f</code>	로그를 실시간으로 확인
<code>docker compose top</code>	실행 중인 컨테이너 프로세스 확인

예시) `docker compose logs -f db`

db 서비스의 로그만 확인

중지 및 삭제 명령어

<code>docker compose stop</code>	컨테이너 중지 (삭제되지 않음)
<code>docker compose start</code>	중지된 컨테이너 다시 시작 (up과 달리 설정을 새로 읽지 않음)
<code>docker compose down</code>	컨테이너 중지 및 컨테이너, 네트워크 삭제
<code>docker compose down -v</code>	컨테이너 중지 및 컨테이너, 네트워크, 볼륨 삭제
<code>docker compose down --rmi all</code>	컨테이너 중지 및 컨테이너, 네트워크, 모든 이미지 삭제
<code>docker compose down --rmi local</code>	컨테이너 중지 및 컨테이너, 네트워크, compose가 직접 빌드한 이미지만 삭제
<code>docker compose down -v --rmi all</code>	컨테이너 중지 및 컨테이너, 네트워크, 볼륨, 모든 이미지 삭제

예 시) `docker compose stop db`

db 서비스만 중지

실행 컨테이너 조작

`docker compose exec <서비스명> <명령어>`

실행 컨테이너 내부에서 명령어 실행

`docker compose restart`

서비스 재시작 (설정 변경사항 미반영)

예 시) `docker compose exec db bash`

db 서비스에서 실행



services

```
services:
  my-app:
    image: myapp:db
    container_name: my-app
```

- 구동할 컨테이너를 정의하는 속성으로 가장 중요하며, 반드시 작성되어야 한다.
- 서비스의 이름을 가장 먼저 작성하고, 그 하위에 문법에 따른 속성을 정의한다.
- image는 컨테이너로 실행할 이미지이다.
 - 이미 생성된 이미지이거나, build 속성과 함께 작성하여 생성할 이미지를 정의한다.
- container_name은 생성할 컨테이너의 이름이며 생략 가능하다.
 - docker run --name 옵션과 유사하다.



services

```
build:  
  context: /some-path/my-app  
  dockerfile: Dockerfile
```

- build는 빌드와 관련된 속성을 정의하는 항목이다.
- docker build 명령어에서 제공하는 옵션의 대부분을 지원한다.
 - <https://docs.docker.com/compose/compose-file/build/#build-definition>
- context는 빌드 컨텍스트를 의미한다.
- 상대 경로로 작성 가능하나, 절대 경로로 작성하는 것이 더 명확할 수 있다.
- dockerfile은 빌드할 Dockerfile 이름이다.
 - docker build -f 옵션과 유사하다.



services

```
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "todo_user", "-ptodo_pass"]
  interval: 5s
  timeout: 5s
  retries: 10
```

- 컨테이너 내부에서 주기적으로 명령을 실행해 서비스가 실제로 정상 동작 중인지 검사한다.
- test에 검사 명령어를 지정할 수 있으며, 종료 코드가 0이면 healthy, 아니면 unhealthy이다.
- interval, timeout, retries, start_period로 검사 동작을 세부적으로 조정할 수 있다.
- docker ps 출력의 STATUS 칼럼에 healthy, unhealthy로 표시되며 depends_on의 service_healthy 조건과도 연동된다.



services

```
depends_on:  
  db:  
    condition: service_healthy
```

- 서비스 간 시작 순서와 의존 관계를 정의하는 설정으로, 의존 대상을 먼저 띄운 뒤 해당 서비스를 시작한다.
- 서버와 DB 조합에서는 필수적으로, 서버보다 DB가 먼저 실행되어 서비스가 정상적으로 동작할 수 있도록 보장
- condition 값은 세 가지 중 선택할 수 있다.
 - service_started : 시작만 되면 가능
 - service_healthy : health check 통과되면 가능 (의존 대상에 healthcheck가 정의되어 있어야 동작)
 - service_completed_successfully : 정상 종료까지 대기



services

```
ports:
  - "3000:3000"
environment:
  - MY_DB_HOST=my-db
networks:
  - elice-network
```

- ports는 포트를 노출하기 위해 사용하는 속성으로 docker run -p, -P 옵션과 유사하다.
- environment는 환경 변수를 정의하는 속성으로 docker run -e 옵션과 유사하다.
- networks는 컨테이너의 네트워크를 정의하는 속성으로 docker run --network 옵션과 유사하다.



services

```
volumes:
  - /some-path/compose/init.sql:/docker-entrypoint-initdb.d/init.sql
env_file:
  - .mysql-env
command:
  - --default-authentication-plugin=mysql_native_password
```

- volumes는 볼륨을 정의하는 속성으로 docker run 의 -v 옵션과 유사하다.
 - 예시에서는 bind-mount 방식을 사용했다.
- command는 Dockerfile의 CMD 명령어와 같다.
 - 예시에서는 ENTRYPOINT의 파라미터 역할로 사용되었다.



network

```
networks:  
  elice-network:
```

- 도커 컴포넌트 중 하나인 도커 네트워크를 정의하는 항목이다.
- 네트워크의 이름을 가장 먼저 작성하고, 그 하위에 문법에 따른 속성을 정의한다.
 - <https://docs.docker.com/compose/compose-file/06-networks/>
- docker network create 명령어와 마찬가지로 기본은 bridge 드라이버 타입의 네트워크가 생성된다.



ports와 expose

- 앞서 학습한 Dockerfile의 EXPOSE 명령어와 docker run -p 옵션의 관계와 유사하다.
- expose 속성은 컨테이너가 노출할 포트에 대한 명세로서의 역할만 가진다.
 - Dockerfile의 EXPOSE 명령어와 비슷하다.
- 추가로, 호스트 머신 또는 외부 네트워크에 노출되지 않는 것을 권장한다는 의미도 가진다.
 - Dockerfile의 EXPOSE 명령어와의 차별점이다.
- ports 속성은 컨테이너가 실제로 노출할 포트 정보이며, 기능적 역할을 가진다.
 - docker run -p 옵션과 유사하다.
- ports 속성은 다양한 형태로 표현할 수 있으며, 크게 짧은 구문과 긴 구문으로 나뉜다.



ports의 다양한 표기 방법 - 짧은 구문

- 짧은 구문의 주요 사용 예시는 다음과 같다.

```
ports:
  # 컨테이너의 3000/tcp 포트를 호스트 머신의 임시 포트와 바인딩
  - "3000"
  # 컨테이너의 3000/tcp 포트를 호스트 머신의 9000 포트와 바인딩
  - "9000:3000"
  # 컨테이너의 6개(3000/tcp, 3001/tcp, ..., 3005/tcp)의 포트를 호스트 머신의 임시 포트 6개와 각각 바인딩
  - "3000-3005"
  # 컨테이너의 6개(3000/tcp, 3001/tcp, ..., 3005/tcp)의 포트를 호스트 머신의 6개(9000, 9001, ..., 9005)포트와 각각 바인딩
  - "9000-9005:3000-3005"
  # 컨테이너의 3000 포트를 호스트 머신의 6개(9000, 9001, ..., 9005) 포트 중 임의의 1개 포트와 바인딩
  - "9000-9005:3000"
  # 예러 ** 컨테이너 포트가 범위인 경우 호스트 머신의 포트와 갯수가 같아야한다.*
  - "9000-9005:3000-3001"
  # 컨테이너의 3000 포트를 127.0.0.1 호스트의 임시 포트와 바인딩
  - "127.0.0.1::3000"
  # 컨테이너의 3000 포트를 127.0.0.1 호스트의 9001 포트와 바인딩
  - "127.0.0.1:9001:3000"
  # 컨테이너의 3000/udp 포트를 호스트 머신의 9000 포트와 바인딩
  - "9000:3000/udp"
  # 예러
  - "9000/udp:3000/udp"
```



ports의 다양한 표기 방법 – 긴 구문

- 긴 구문은 3.2 버전부터 지원한다.
- 각 항목을 구체적으로 명세할 수 있다.

```
ports:
  # 컨테이너의 포트
  - target : 3000
    # 호스트 또는 외부 네트워크의 포트
  - published : 3000
    # tcp 또는 udp
  - protocol : tcp
    # host 또는 ingress(swarm mode)
  - mode : host
    # host ip
  - host_ip : 127.0.0.1
```

Web Basic

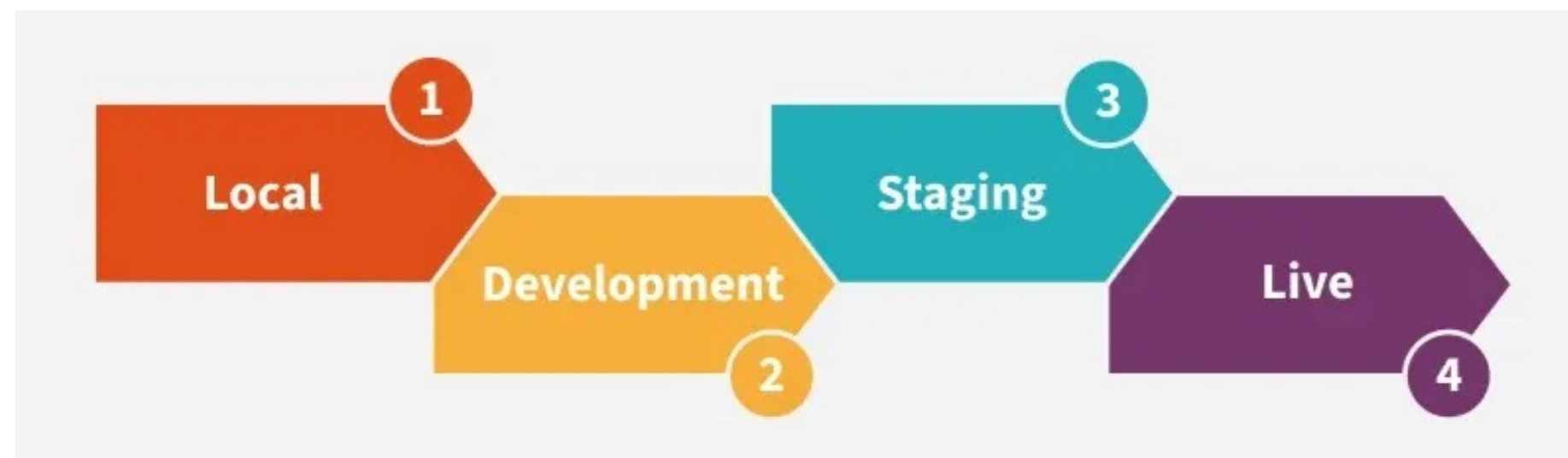
Dockerfile과 Docker Compose

배포 환경

배포 환경

배포란?

- 개발자가 자신의 컴퓨터에서 만든 소프트웨어를 실제로 사용자들이 사용할 수 있도록 서버나 클라우드 같은 환경에 올리는 과정이다.
- 내 컴퓨터에서는 오류가 없더라도, 수천명이 접속하는 서버 환경에서는 예상치 못한 오류가 발생할 수 있다.
- 따라서, 환경의 일관성을 유지하여 안정적으로 서비스를 운영하는 것이 배포의 핵심이다.
- 도커는 환경의 일관성을 유지하는 도구라고 할 수 있다.





개발 환경과 배포 환경의 차이

- 서비스를 개발할 때 중요한 것들과 배포할 때 중요한 것들은 다르다.
- 서비스 개발 시 주요 개발 환경*은 로컬(local), 개발(dev), 운영 (prod)로 구분된다.
 - 로컬 환경은 개별 개발자의 컴퓨터 환경을 의미하며, 즉각적인 실행 결과를 확인하며 자유롭게 개발하는 공간이다.
 - 개발 환경은 팀 단위로 작성된 코드들이 하나로 합쳐지는 공간으로, 기능 구현 여부를 검증하고 테스트를 진행하는 것이 주 목적이다.
 - 배포 환경은 실제 사용자들이 접속하여 서비스를 이용하는 공간으로 성능, 모니터링, 제한된 자원 최적화 등의 이슈가 중요하다.
- **docker-compose.yml** 파일 관점에서 생각해보면, 환경에 따라 사용하는 속성과 값이 많이 다를 수 있다.



환경에 따라 달라질 수 있는 요소

- 환경 변수는 프로세스 실행에 영향을 미치는 동적인 값이다.
- 따라서 환경에 따라 달라질 가능성이 높다.
- 각 환경 별 도커 호스트가 다르므로 데이터 영속화를 위한 볼륨이 달라진다.
- 네트워크 환경도 달라질 수 있으므로 포트 바인딩이 달라질 수 있다.
- 로그 관리 목적이 다를 수 있다.



docker-compose.yml 분리하기

- 앞서 알아본 것과 같이 두 환경은 서비스 실행을 위해 겹치는 부분과 다른 부분이 있다.
- 하나의 docker-compose.yml 로 관리하기에는 실수가 발생할 수 있고, 계속 시간을 할애해야 할 수 있다.
- 따라서 docker-compose.yml 파일을 공통된 부분과 다른 부분을 나누어 관리하는 것이 편리하다.
- my-app의 환경변수 MY_DB_HOST가 배포 환경에서는 my-db-prod 값으로 주입되어야 한다고 가정하자.
- production.yml 파일을 아래와 같이 작성 후 -f 옵션으로 다중 파일을 지정하면 환경 변수를 덮어쓸 수 있다.

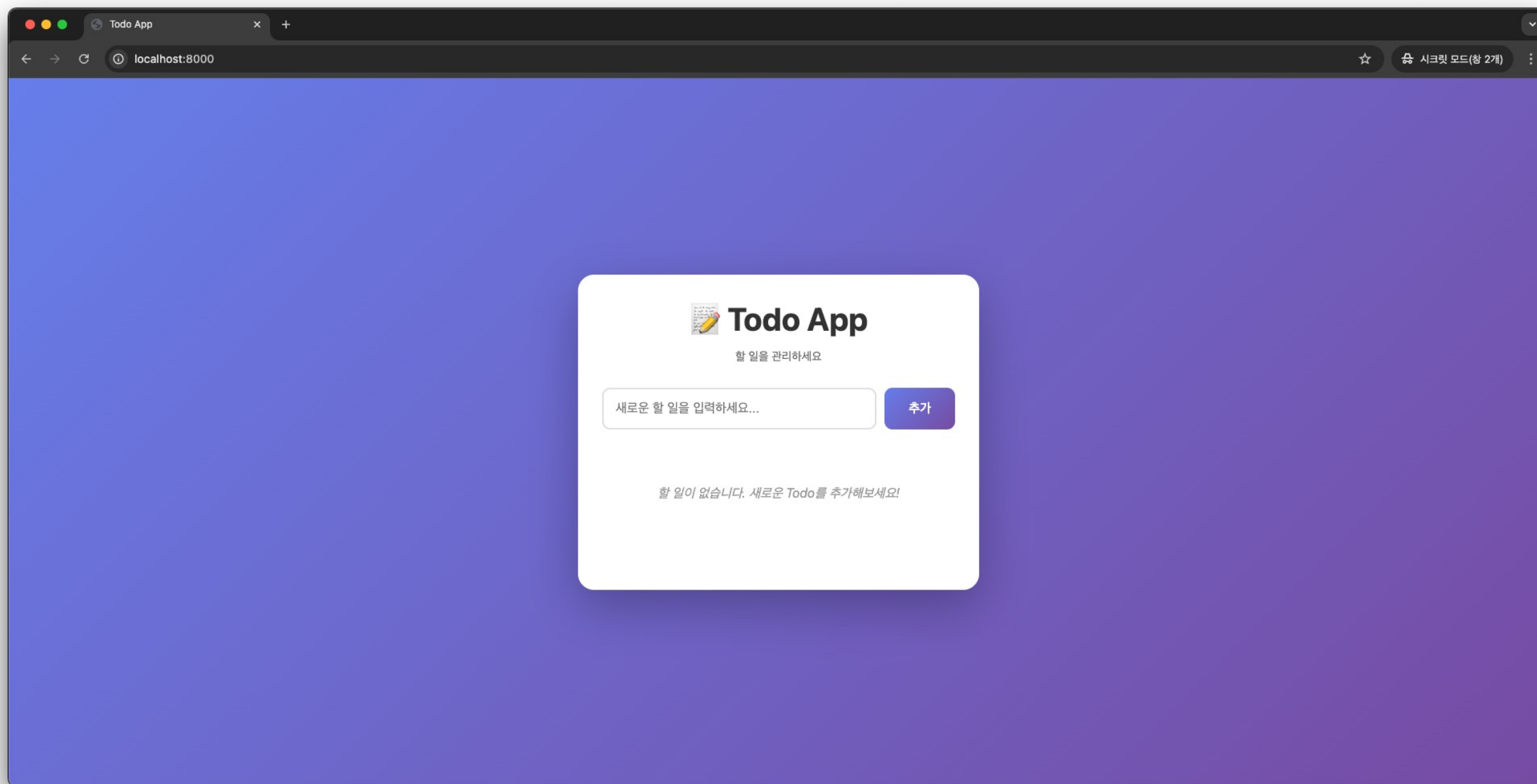
```
version: "3.8"
services:
  my-app:
    environment:
      - MY_DB_HOST=my-db-prod
# 다중 파일 실행
# docker compose -f docker-compose.yml -f production.yml up -d
```


FastAPI 웹 애플리케이션 Docker Compose 실습



[실습] 간단한 서비스 구성하기

[실습] 간단한 FastAPI 웹 애플리케이션 Docker Compose 구성하기

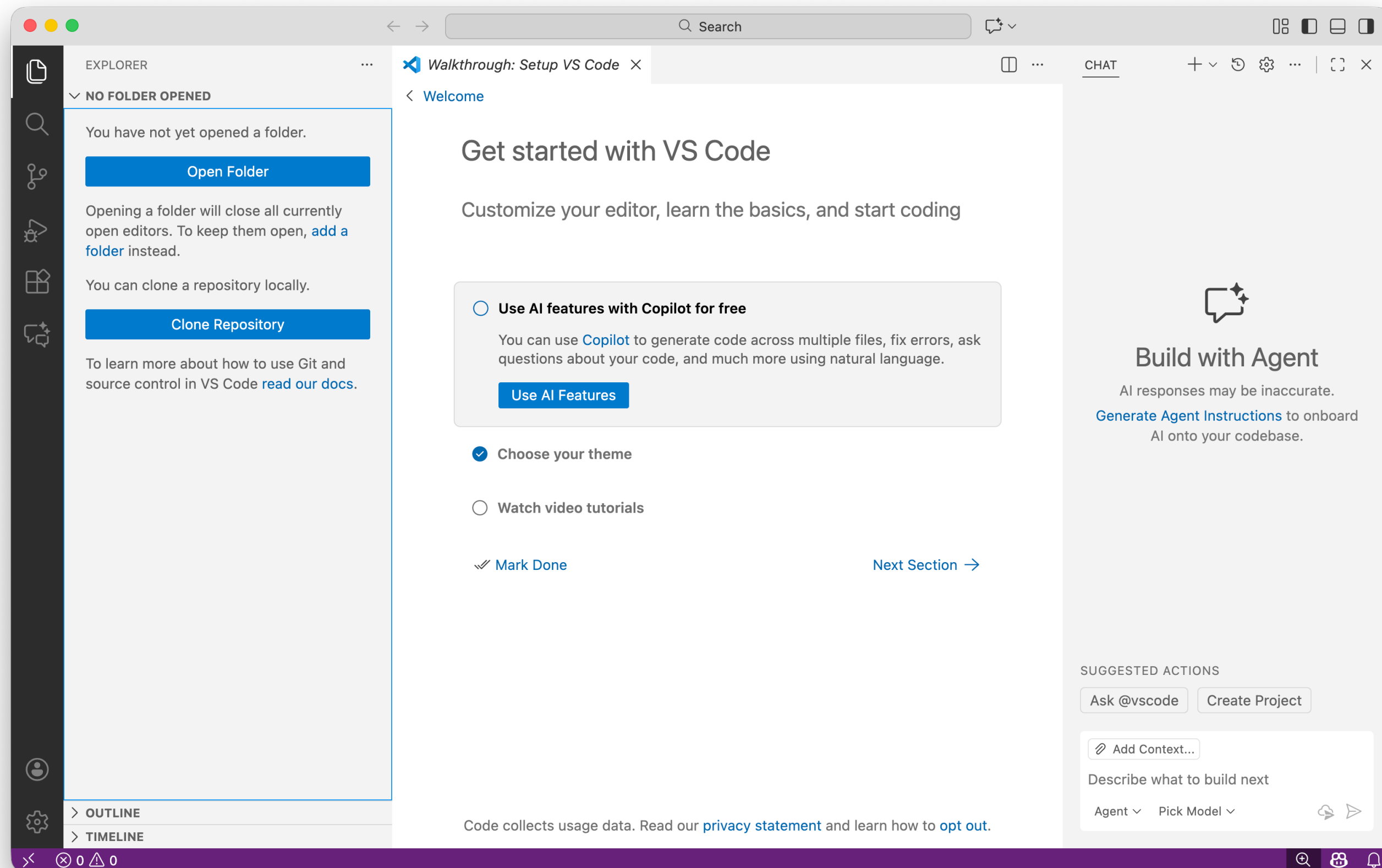


앞 실습을 통해 단일 서비스
환경(Dockerfile 작성 → 이미지 빌드
→ 컨테이너 실행)을 이해한 후,
이번 실습에서는 docker-
compose를 활용하여 웹 서비스와
데이터베이스를 하나의 구성으로
통합하고 동시에 실행하는 과정을
실습합니다.



[실습] 간단한 서비스 구성하기

[실습] 간단한 FastAPI 웹 애플리케이션 Docker Compose 구성하기



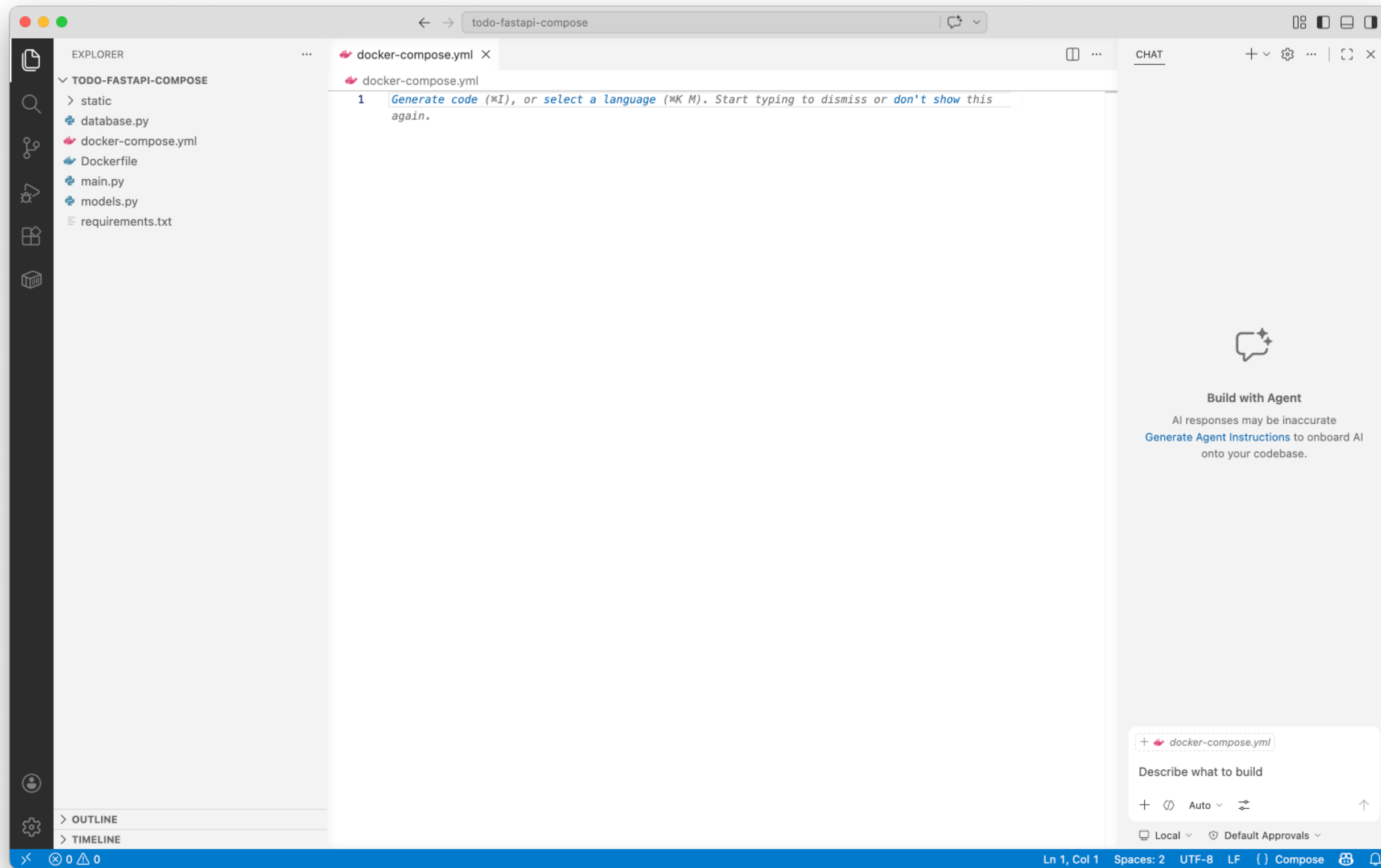
1. Open Folder 클릭

2. 사전에 제공된 Docker 파일 열기



[실습] 간단한 서비스 구성하기

[실습] 간단한 FastAPI 웹 애플리케이션 Docker Compose 구성하기



1. 우측 상단 파일 아이콘 클릭
2. 파일명 docker-compose.yml 입력
3. docker-compose.yml 작성



[실습] 간단한 FastAPI 웹 애플리케이션 Docker Compose 구성하기

```
services:
  db:
    image: mysql:8.0
    container_name: todo-db
    environment:
      MYSQL_USER: todo_user
      MYSQL_PASSWORD: todo_pass
      MYSQL_DATABASE: todo_db
      MYSQL_ROOT_PASSWORD: root_pass
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "todo_user", "-ptodo_pass"]
      interval: 5s
      timeout: 5s
      retries: 10

  web:
    build: .
    container_name: todo-web
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: mysql+pymysql://todo_user:todo_pass@db:3306/todo_db
    depends_on:
      db:
        condition: service_healthy

volumes:
  mysql_data:
```

- ✓ **services** : 실행할 서비스를 구분하여 작성
- ✓ **environment** : 각 컨테이너에서 사용할 환경변수 작성
- ✓ **ports** : 호스트와 컨테이너의 포트를 연결해 외부에서 컨테이너 접근을 허용하는 설정
- ✓ **volume** : 호스트 경로나 명명된 볼륨을 컨테이너 내부 경로에 마운트해 데이터를 영구 저장
- ✓ **healthcheck** : 컨테이너 내부에서 주기적으로 명령을 실행해 실제로 동작하는지 확인하는 설정
- ✓ **depends_on** : 서비스 간 시작 순서와 의존 관계를 정의하는 설명으로, 의존 대상의 healthcheck 통과까지 대기